

Docker final task

Goal of the Exercise

The goal of this exercise is to test your understanding of a Cloud Compute deployment with Terraform.

You are tasked with creating a codebase for a microservice deployment on a cloud environment, using **Terraform**. Since our focus is not the application itself, the service will be a simple web server that returns a static page.

Requirements

Service Details

The service will be a simple web server that returns a static page.

You can use **apache2** or **nginx** to serve a simple HTML file.

The service should be **containerized** and stored in a **Docker registry**.

The service should be deployed on a **single VM instance** as a docker container.

The image should be pulled from the registry specified earlier.

- The service should **not** be accessible from the internet but only through a **Load Balancer**, except for SSH access from specific IP addresses.

Deliverables

You are to provide the following as part of your solution:

1. **Dockerfile** to build the image.
2. **Terraform codebase** to deploy the following:
3. Docker registry
4. VPC and all required networking resources
5. EC2 instance
6. Load Balancer

Notes

Since a CI/CD pipeline is not part of this exercise, we will work around that with the help of Terraform.

To push the image to the Docker registry, **build the image locally** before running the Terraform code.

Once the image registry is created, use Terraform to **push the local image** to the registry.

Similarly, use Terraform to **pull the image** from the registry and run it on the VM instance once it is created.

For SSH access:

Use a locally created key.

Add it to the cloud provider key storage.

Attach it to the instance.

Use **Terraform best practices** for file structure and code organization.

Use variables, locals, modules, etc. when appropriate.

Include all **tfvars**, scripts, and other helper files used to deploy the application.

You may use any cloud provider of your choosing, although **AWS is preferred**.

- Use a **free tier account** or a **trial account** to deploy the resources.

—->steps:

Step: 1 create a structure like below tree

(<https://github.com/Waseema761/docker-terraform.git>)

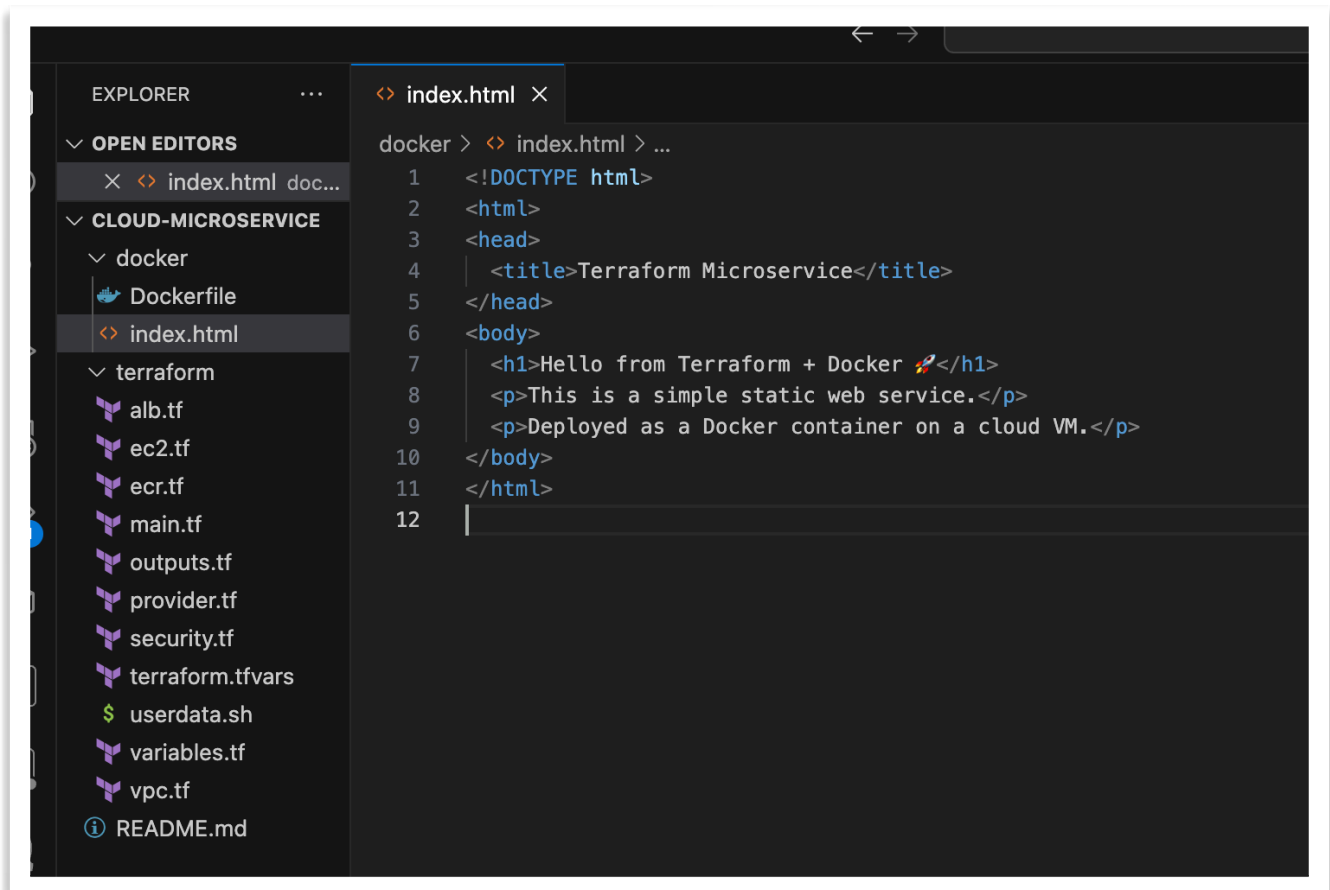
```
cloud-microservice
├── docker
│   ├── Dockerfile
│   └── index.html
├── README.md
└── terraform
    ├── alb.tf
    ├── ec2.tf
    ├── ecr.tf
    ├── main.tf
    ├── outputs.tf
    ├── provider.tf
    ├── security.tf
    ├── terraform.tfvars
    ├── userdata.sh
    ├── variables.tf
    └── vpc.tf
```

Now open vs code and add details of each file manually one by one

STEP 2: Docker Code

docker/index.html

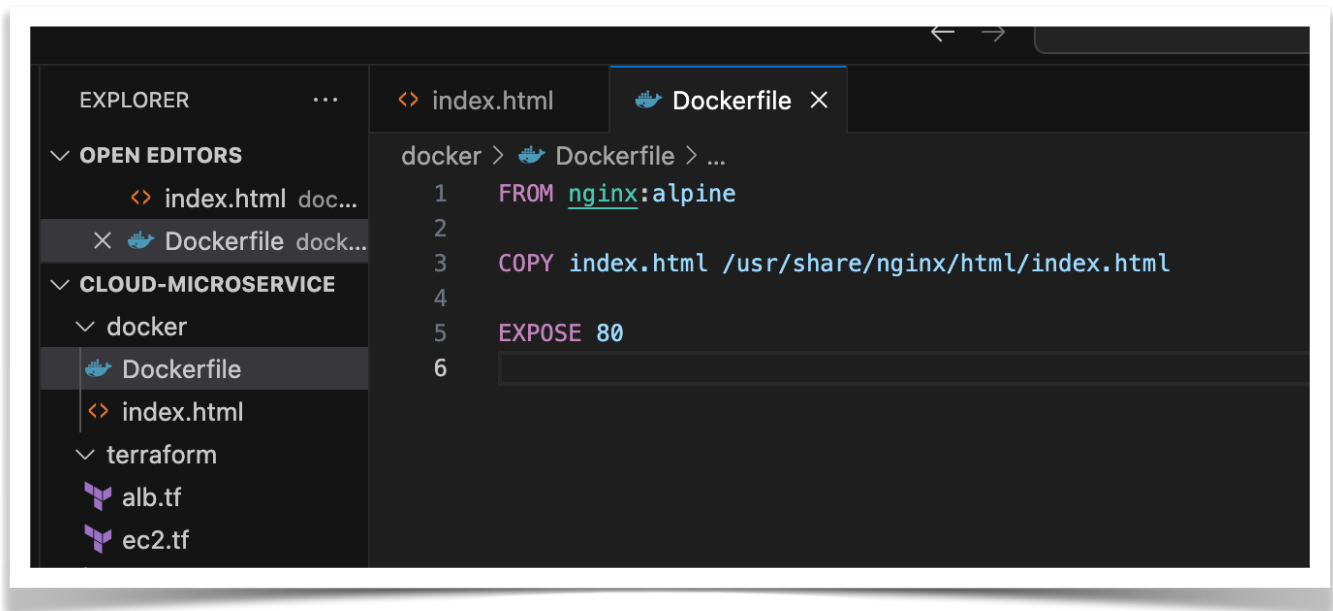
Paste this:

A screenshot of the Visual Studio Code editor interface. The Explorer sidebar on the left shows a project structure with folders 'OPEN EDITORS', 'CLOUD-MICROSERVICE', 'docker', and 'terraform'. The 'docker' folder is expanded, showing 'Dockerfile' and 'index.html'. The 'index.html' file is selected and its content is displayed in the main editor area. The code is an HTML document with a title 'Terraform Microservice' and a body containing a welcome message and deployment information.

```
1 <!DOCTYPE html>
2 <html>
3 <head>
4   <title>Terraform Microservice</title>
5 </head>
6 <body>
7   <h1>Hello from Terraform + Docker 🚀</h1>
8   <p>This is a simple static web service.</p>
9   <p>Deployed as a Docker container on a cloud VM.</p>
10 </body>
11 </html>
12
```

docker/Dockerfile

Paste this:



Save both files.

docker/

└─ Dockerfile

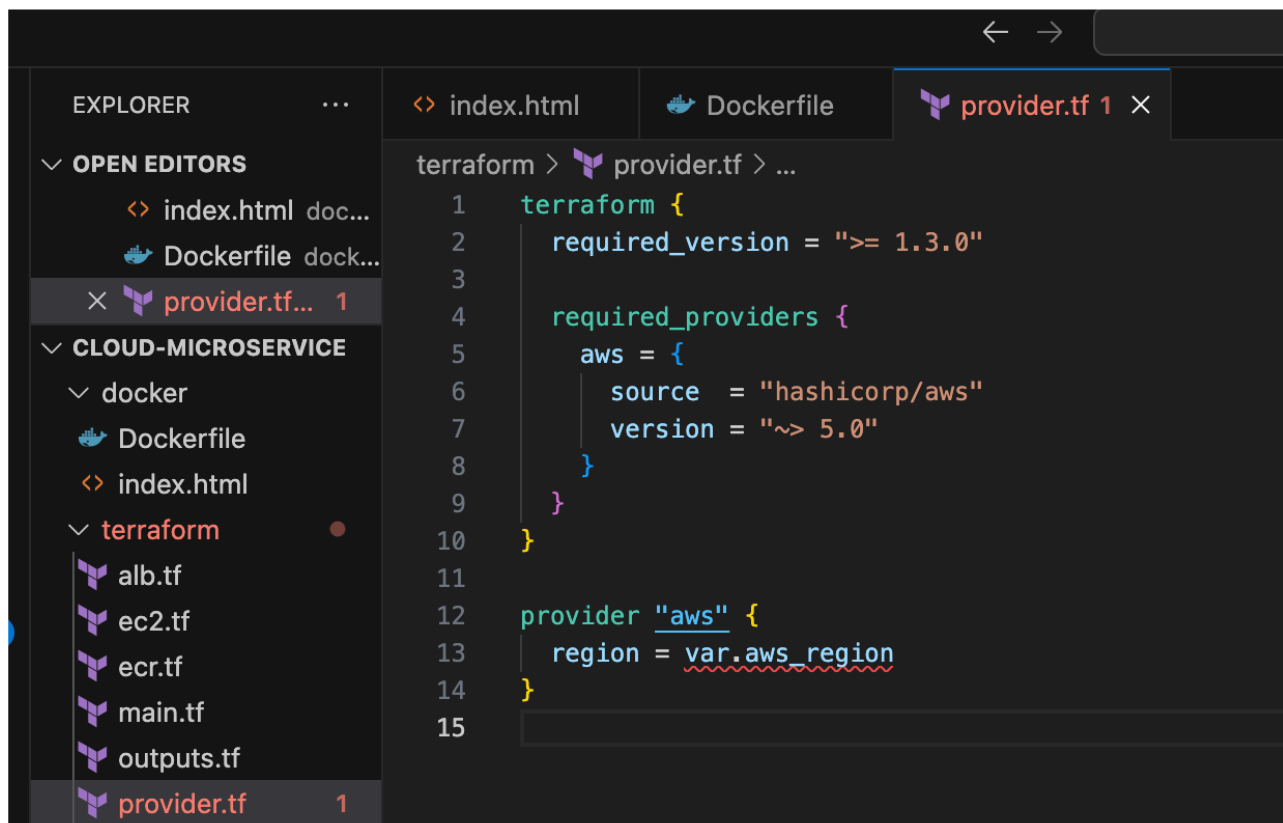
└─ index.html

✓ Docker part DONE.

Step:3

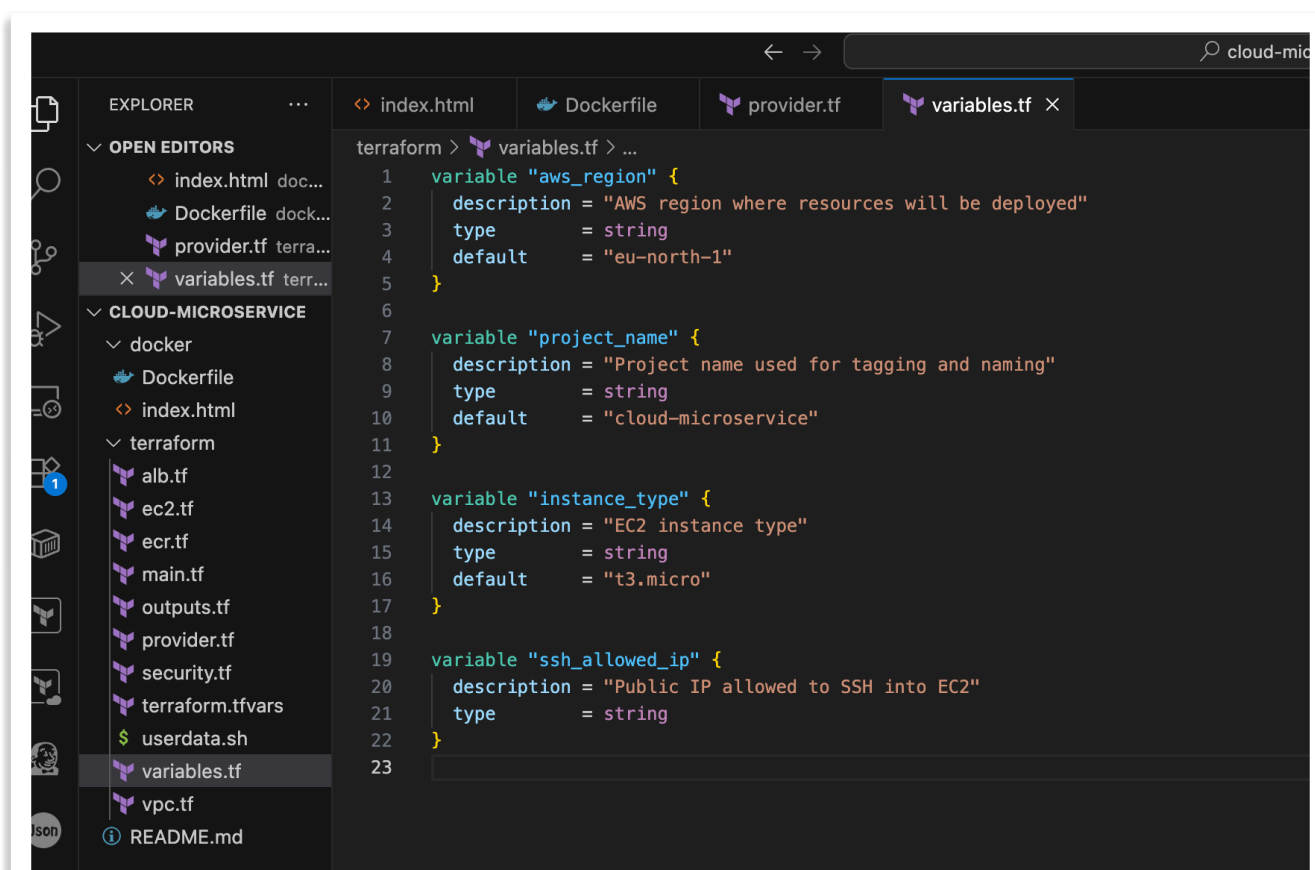
terraform/provider.tf

Open **terraform/provider.tf** and paste:



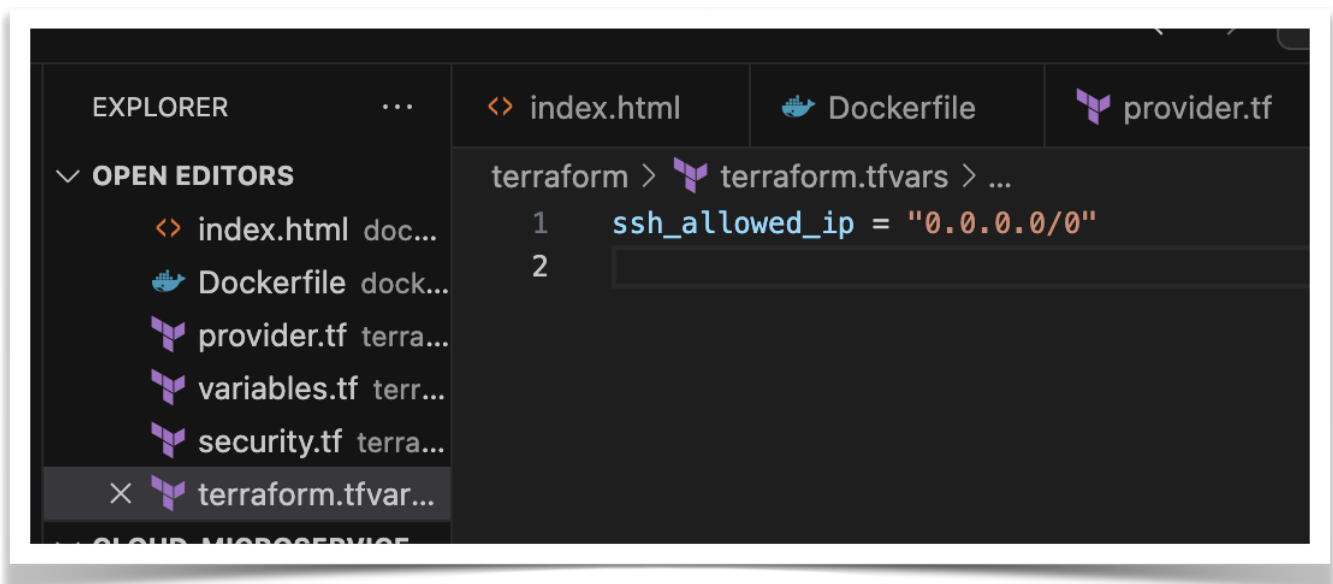
terraform/variables.tf

Open **terraform/variables.tf** and paste:



Open this file:

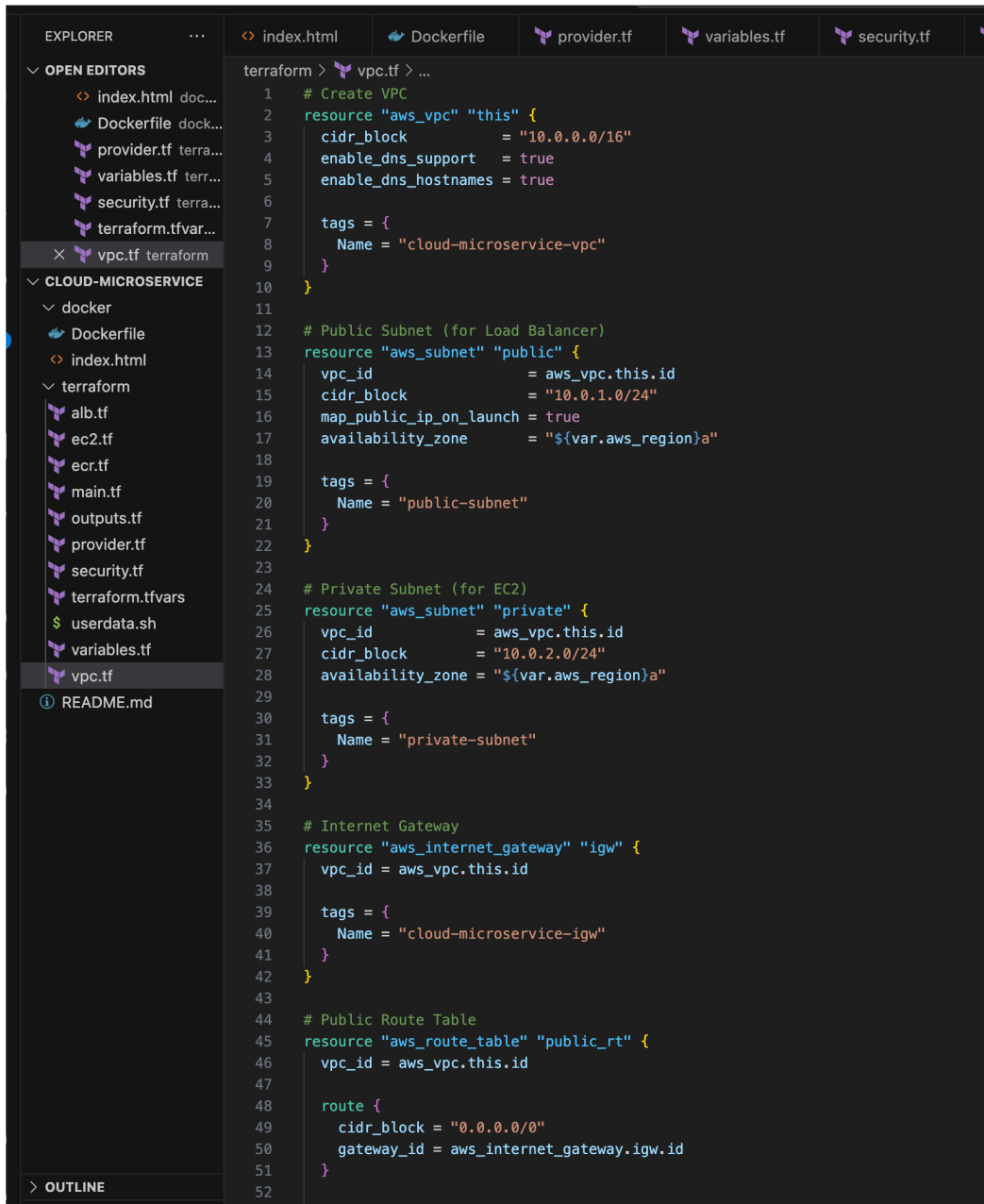
terraform/terraform.tfvars



STEP 4: VPC and Networking

terraform/vpc.tf

Open **terraform/vpc.tf** and paste everything below



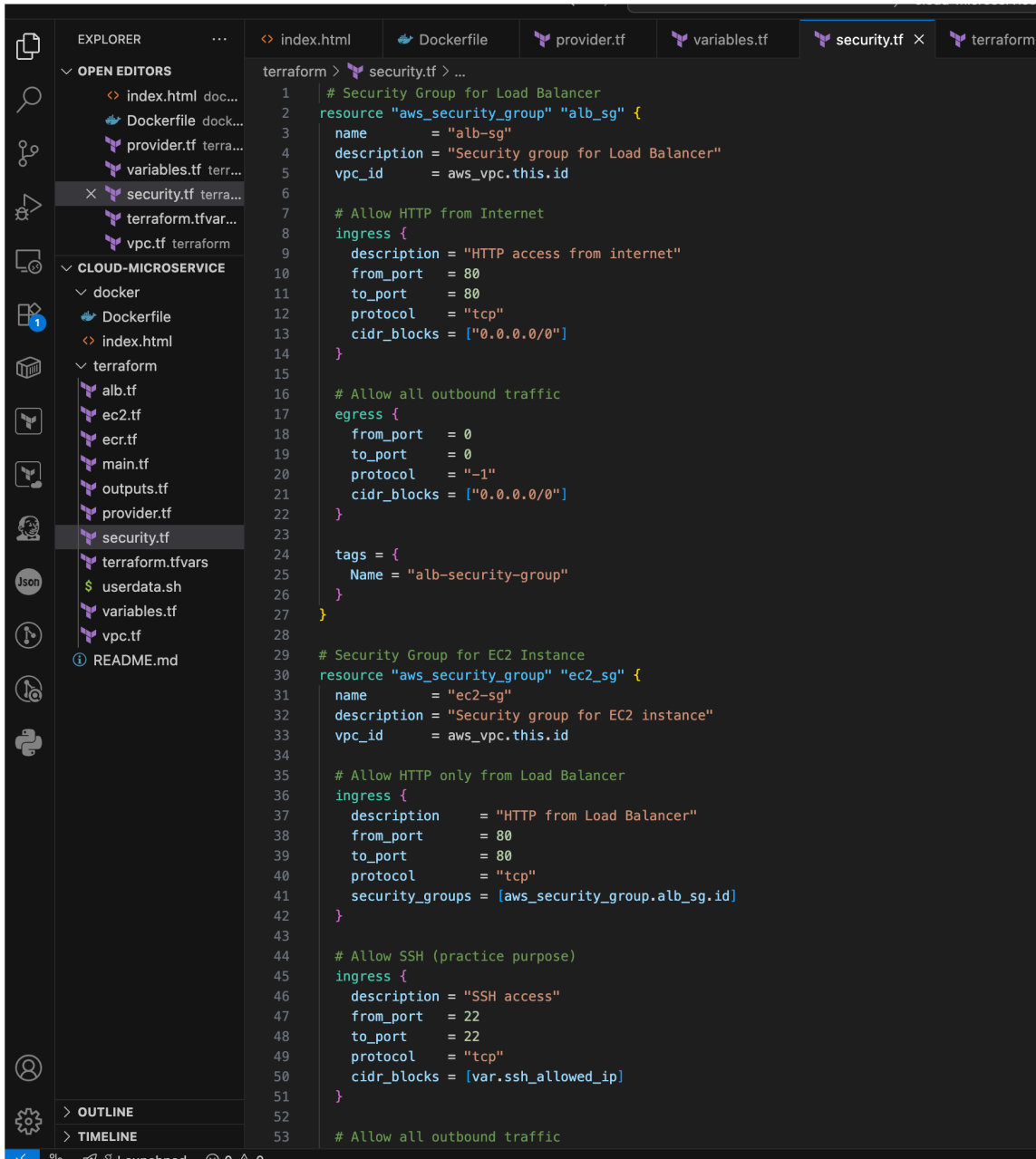
The screenshot shows a code editor with a file explorer on the left and a code editor on the right. The file explorer shows a project structure with files like index.html, Dockerfile, provider.tf, variables.tf, security.tf, terraform.tfvars, vpc.tf, alb.tf, ec2.tf, ecr.tf, main.tf, outputs.tf, provider.tf, security.tf, terraform.tfvars, userdata.sh, variables.tf, and README.md. The code editor shows the content of vpc.tf, which defines an AWS VPC and three subnets (public, private, and internet gateway) using Terraform syntax.

```
1 # Create VPC
2 resource "aws_vpc" "this" {
3   cidr_block      = "10.0.0.0/16"
4   enable_dns_support = true
5   enable_dns_hostnames = true
6
7   tags = {
8     Name = "cloud-microservice-vpc"
9   }
10 }
11
12 # Public Subnet (for Load Balancer)
13 resource "aws_subnet" "public" {
14   vpc_id            = aws_vpc.this.id
15   cidr_block        = "10.0.1.0/24"
16   map_public_ip_on_launch = true
17   availability_zone  = "${var.aws_region}a"
18
19   tags = {
20     Name = "public-subnet"
21   }
22 }
23
24 # Private Subnet (for EC2)
25 resource "aws_subnet" "private" {
26   vpc_id            = aws_vpc.this.id
27   cidr_block        = "10.0.2.0/24"
28   availability_zone  = "${var.aws_region}a"
29
30   tags = {
31     Name = "private-subnet"
32   }
33 }
34
35 # Internet Gateway
36 resource "aws_internet_gateway" "igw" {
37   vpc_id = aws_vpc.this.id
38
39   tags = {
40     Name = "cloud-microservice-igw"
41   }
42 }
43
44 # Public Route Table
45 resource "aws_route_table" "public_rt" {
46   vpc_id = aws_vpc.this.id
47
48   route {
49     cidr_block = "0.0.0.0/0"
50     gateway_id = aws_internet_gateway.igw.id
51   }
52 }
```

Simple Explanation

- **VPC** → your private network
- **Public subnet** → Load Balancer lives here
- **Private subnet** → EC2 lives here
- **Internet Gateway** → internet access
- **Route table** → tells public subnet how to reach internet

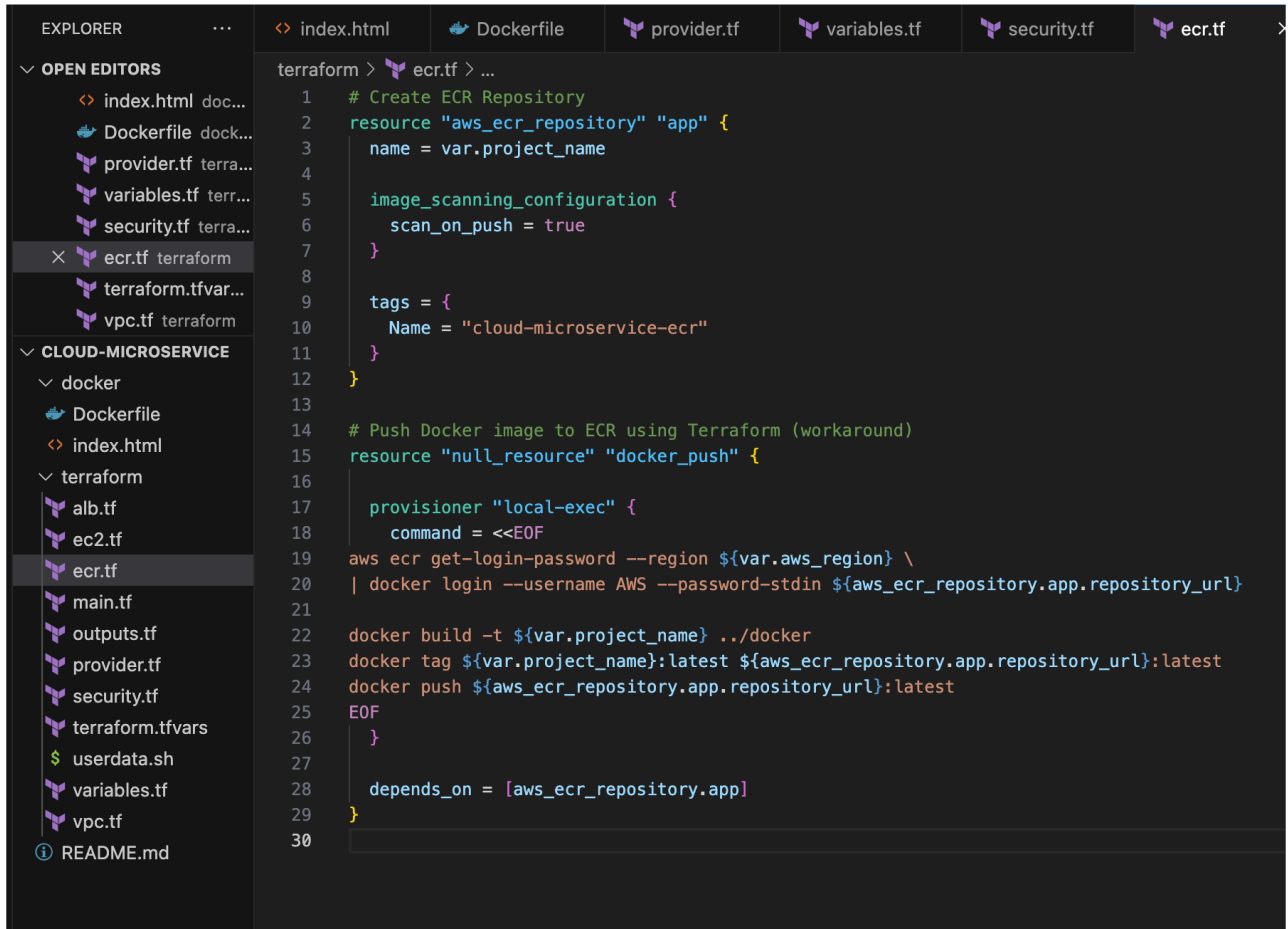
terraform/security.tf



```
1  # Security Group for Load Balancer
2  resource "aws_security_group" "alb_sg" {
3      name           = "alb-sg"
4      description    = "Security group for Load Balancer"
5      vpc_id         = aws_vpc.this.id
6
7      # Allow HTTP from Internet
8      ingress {
9          description = "HTTP access from internet"
10         from_port   = 80
11         to_port     = 80
12         protocol    = "tcp"
13         cidr_blocks = ["0.0.0.0/0"]
14     }
15
16     # Allow all outbound traffic
17     egress {
18         from_port   = 0
19         to_port     = 0
20         protocol    = "-1"
21         cidr_blocks = ["0.0.0.0/0"]
22     }
23
24     tags = {
25         Name = "alb-security-group"
26     }
27 }
28
29 # Security Group for EC2 Instance
30 resource "aws_security_group" "ec2_sg" {
31     name           = "ec2-sg"
32     description    = "Security group for EC2 instance"
33     vpc_id         = aws_vpc.this.id
34
35     # Allow HTTP only from Load Balancer
36     ingress {
37         description = "HTTP from Load Balancer"
38         from_port   = 80
39         to_port     = 80
40         protocol    = "tcp"
41         security_groups = [aws_security_group.alb_sg.id]
42     }
43
44     # Allow SSH (practice purpose)
45     ingress {
46         description = "SSH access"
47         from_port   = 22
48         to_port     = 22
49         protocol    = "tcp"
50         cidr_blocks = [var.ssh_allowed_ip]
51     }
52
53     # Allow all outbound traffic
```

STEP 5: ECR (Docker Registry)

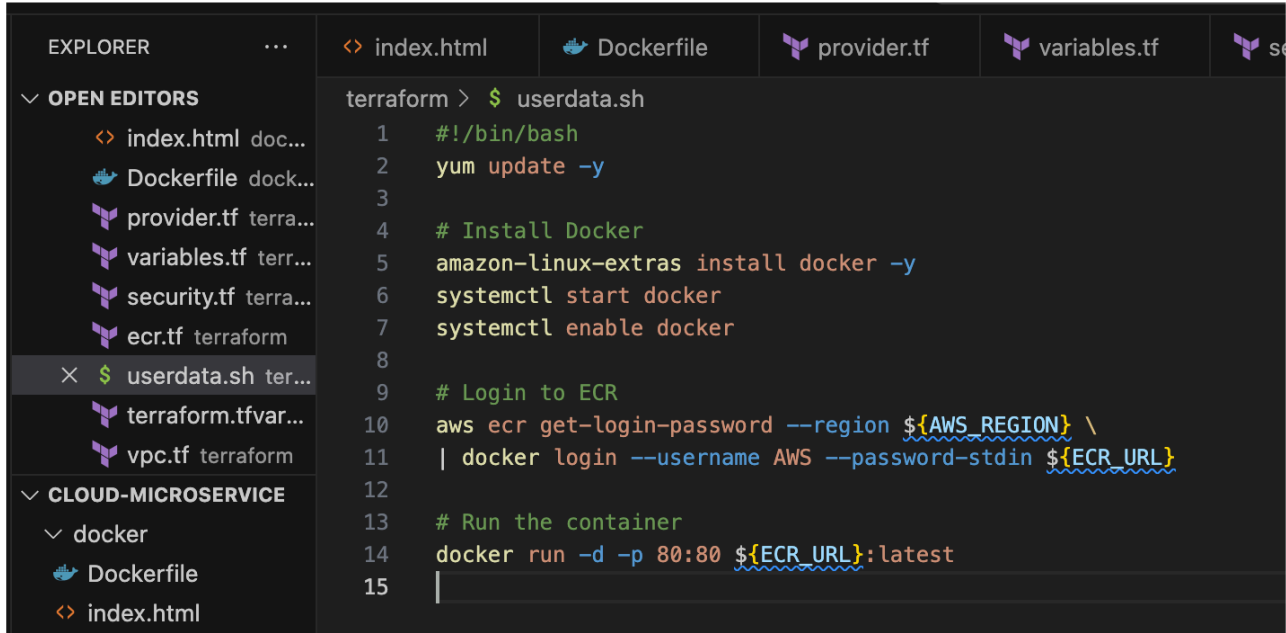
 terraform/ecr.tf



```
terraform > ecr.tf > ...
1  # Create ECR Repository
2  resource "aws_ecr_repository" "app" {
3      name = var.project_name
4
5      image_scanning_configuration {
6          scan_on_push = true
7      }
8
9      tags = {
10         Name = "cloud-microservice-ecr"
11     }
12 }
13
14 # Push Docker image to ECR using Terraform (workaround)
15 resource "null_resource" "docker_push" {
16
17     provisioner "local-exec" {
18         command = <<EOF
19         aws ecr get-login-password --region ${var.aws_region} \
20         | docker login --username AWS --password-stdin ${aws_ecr_repository.app.repository_url}
21
22         docker build -t ${var.project_name} ../docker
23         docker tag ${var.project_name}:latest ${aws_ecr_repository.app.repository_url}:latest
24         docker push ${aws_ecr_repository.app.repository_url}:latest
25     EOF
26     }
27
28     depends_on = [aws_ecr_repository.app]
29 }
30
```

STEP 6: EC2 Instance + User Data

 terraform/userdata.sh



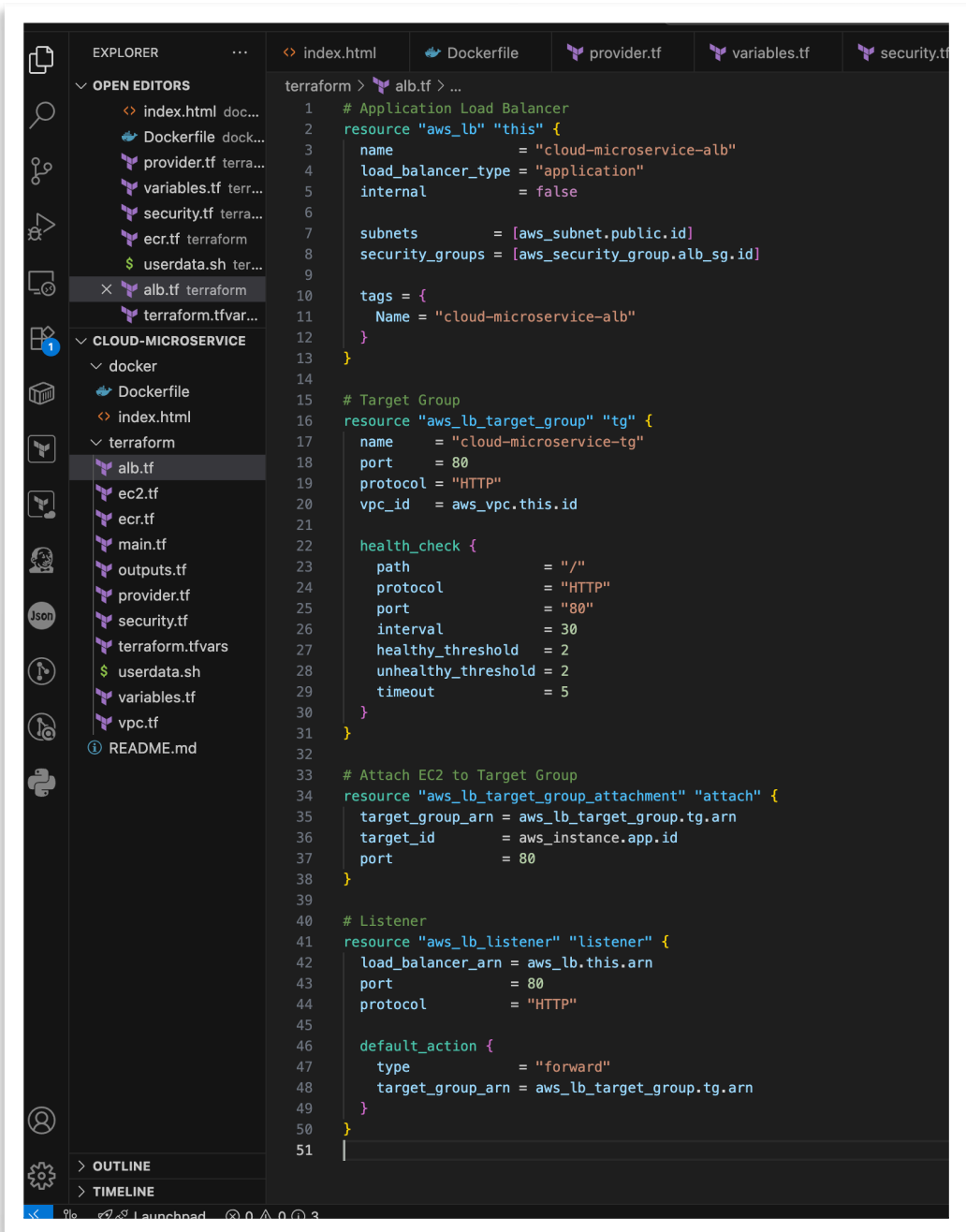
The screenshot shows a code editor with a dark theme. On the left, the 'EXPLORER' sidebar is open, showing a file tree with 'OPEN EDITORS' and 'CLOUD-MICROSERVICE' sections. The 'OPEN EDITORS' section lists several files, including 'index.html', 'Dockerfile', 'provider.tf', 'variables.tf', 'security.tf', 'ecr.tf', and 'userdata.sh'. The 'CLOUD-MICROSERVICE' section shows a 'docker' folder containing 'Dockerfile' and 'index.html'. The main editor area displays the content of 'userdata.sh', which is a shell script for installing Docker and running a container. The script includes comments and uses environment variables for AWS region and ECR URL. The script content is as follows:

```
terraform > $ userdata.sh
1  #!/bin/bash
2  yum update -y
3
4  # Install Docker
5  amazon-linux-extras install docker -y
6  systemctl start docker
7  systemctl enable docker
8
9  # Login to ECR
10 aws ecr get-login-password --region ${AWS_REGION} \
11 | docker login --username AWS --password-stdin ${ECR_URL}
12
13 # Run the container
14 docker run -d -p 80:80 ${ECR_URL}:latest
15 |
```

STEP 7: Load Balancer (ALB)

This connects **internet** → **service** and satisfies the task requirement:

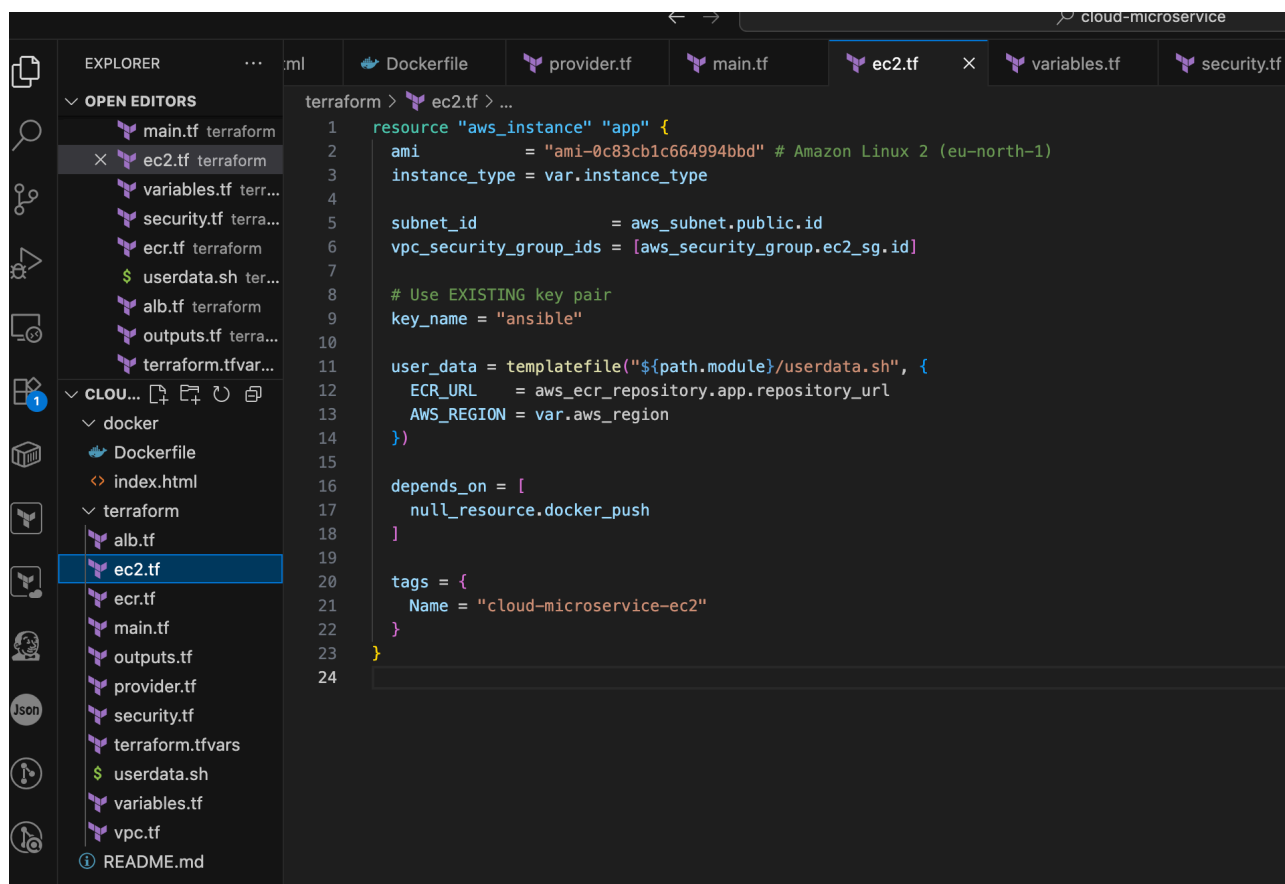
 terraform/alb.tf



terraform/outputs.tf



terraform/ec2.tf



Make sure you have aws—configure

-->

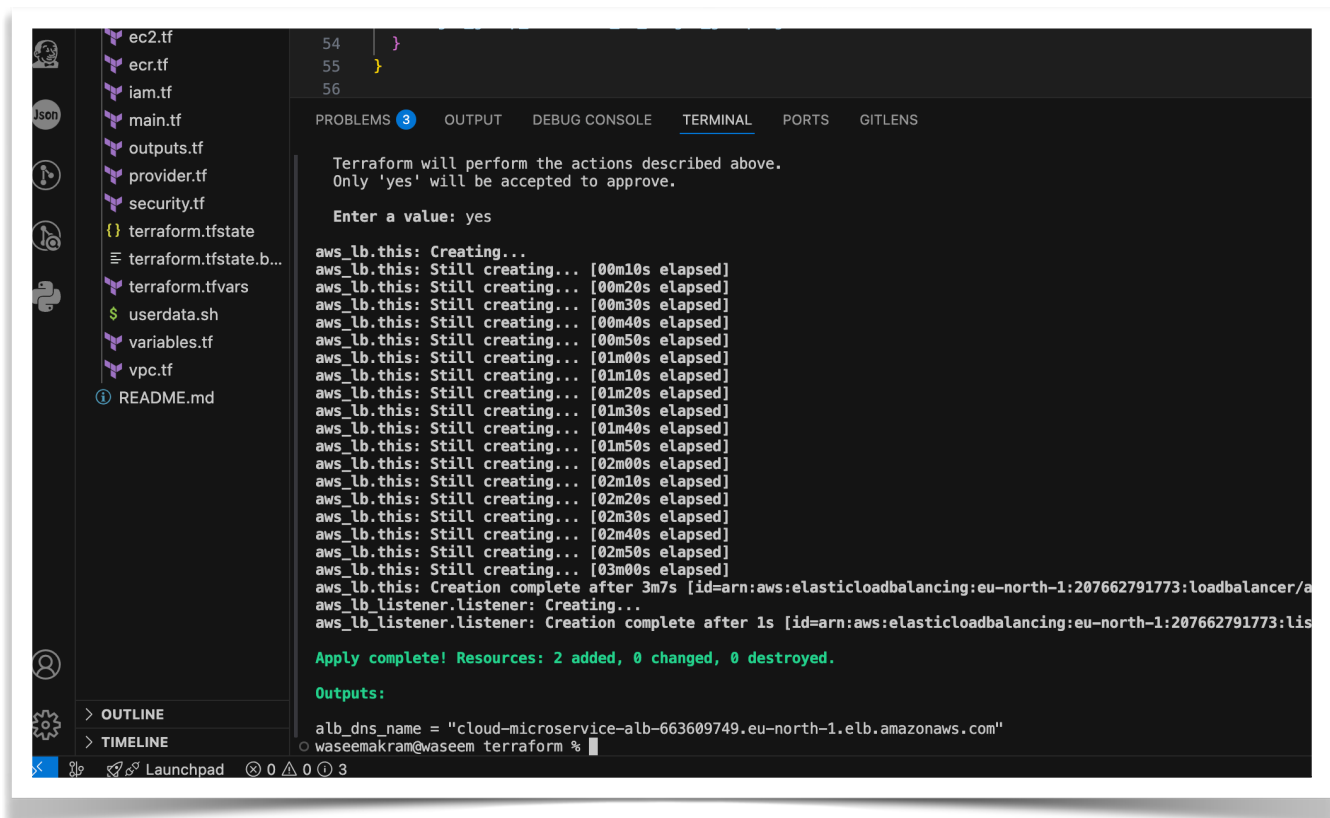
Now in local terminal cd terraform

Cd terraform

Terraform init

Terraform plan

Terraform apply



The screenshot shows a VS Code editor with a file explorer on the left containing Terraform configuration files: ec2.tf, ecr.tf, iam.tf, main.tf, outputs.tf, provider.tf, security.tf, terraform.tfstate, terraform.tfstate.backup, terraform.tfvars, userdata.sh, variables.tf, vpc.tf, and README.md. The terminal window is open to the 'TERMINAL' tab, displaying the output of a Terraform apply command. The output shows the creation of an AWS Elastic Load Balancing listener, with progress updates every 10 seconds. The final status is 'Apply complete! Resources: 2 added, 0 changed, 0 destroyed.' and the output variable 'alb_dns_name' is shown as 'cloud-microservice-alb-663609749.eu-north-1.elb.amazonaws.com'.

```
54 }
55 }
56

Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

aws_lb.this: Creating...
aws_lb.this: Still creating... [00m10s elapsed]
aws_lb.this: Still creating... [00m20s elapsed]
aws_lb.this: Still creating... [00m30s elapsed]
aws_lb.this: Still creating... [00m40s elapsed]
aws_lb.this: Still creating... [00m50s elapsed]
aws_lb.this: Still creating... [01m00s elapsed]
aws_lb.this: Still creating... [01m10s elapsed]
aws_lb.this: Still creating... [01m20s elapsed]
aws_lb.this: Still creating... [01m30s elapsed]
aws_lb.this: Still creating... [01m40s elapsed]
aws_lb.this: Still creating... [01m50s elapsed]
aws_lb.this: Still creating... [02m00s elapsed]
aws_lb.this: Still creating... [02m10s elapsed]
aws_lb.this: Still creating... [02m20s elapsed]
aws_lb.this: Still creating... [02m30s elapsed]
aws_lb.this: Still creating... [02m40s elapsed]
aws_lb.this: Still creating... [02m50s elapsed]
aws_lb.this: Still creating... [03m00s elapsed]
aws_lb.this: Creation complete after 3m7s [id=arn:aws:elasticloadbalancing:eu-north-1:207662791773:loadbalancer/a
aws_lb_listener.listener: Creating...
aws_lb_listener.listener: Creation complete after 1s [id=arn:aws:elasticloadbalancing:eu-north-1:207662791773:lis

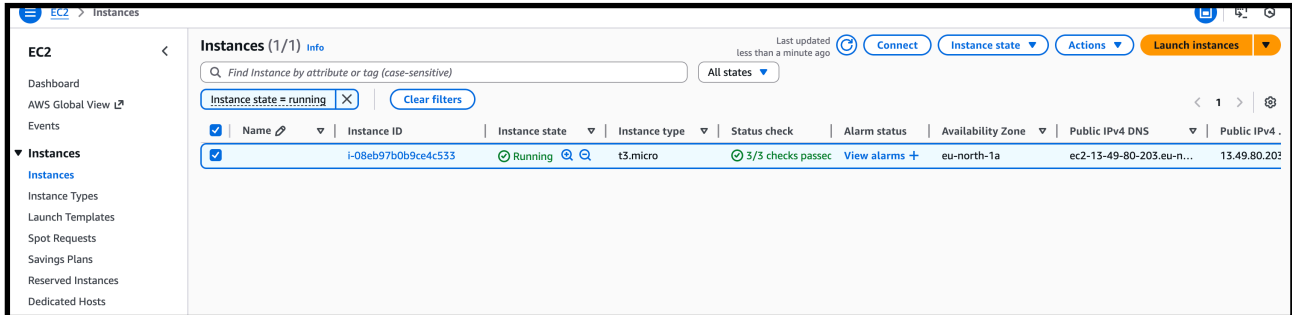
Apply complete! Resources: 2 added, 0 changed, 0 destroyed.

Outputs:

alb_dns_name = "cloud-microservice-alb-663609749.eu-north-1.elb.amazonaws.com"
waseemakram@waseem terraform %
```

Verify:

Ec2



—> docker images

—> **Docker Image & Registry (ECR)**

What to check

- ECR repository exists
- Image pushed successfully

How to verify

```
aws ecr list-images --repository-name cloud-microservice
```

```
[root@ip-10-0-1-197 ~]# aws ecr list-images --repository-name cloud-microservice
{
  "imageIds": [
    {
      "imageDigest": "sha256:b2fb71c80f1d682fcd6dd76601703690c3e23a164b3e6287002eeab9d8be908f"
    },
    {
      "imageDigest": "sha256:ba79e45a48194285fca3497eab6185a0cceb7d735724edde554799003e14ef2e"
    },
    {
      "imageDigest": "sha256:7dbbcb52114eca0fc438ecf049e255674fdc32332a5063474fe53da5aef31b8",
      "imageTag": "latest"
    }
  ]
}
[root@ip-10-0-1-197 ~]#
```

i-071090839d3a5e42a

PublicIPs: 13.61.183.79 PrivateIPs: 10.0.1.197

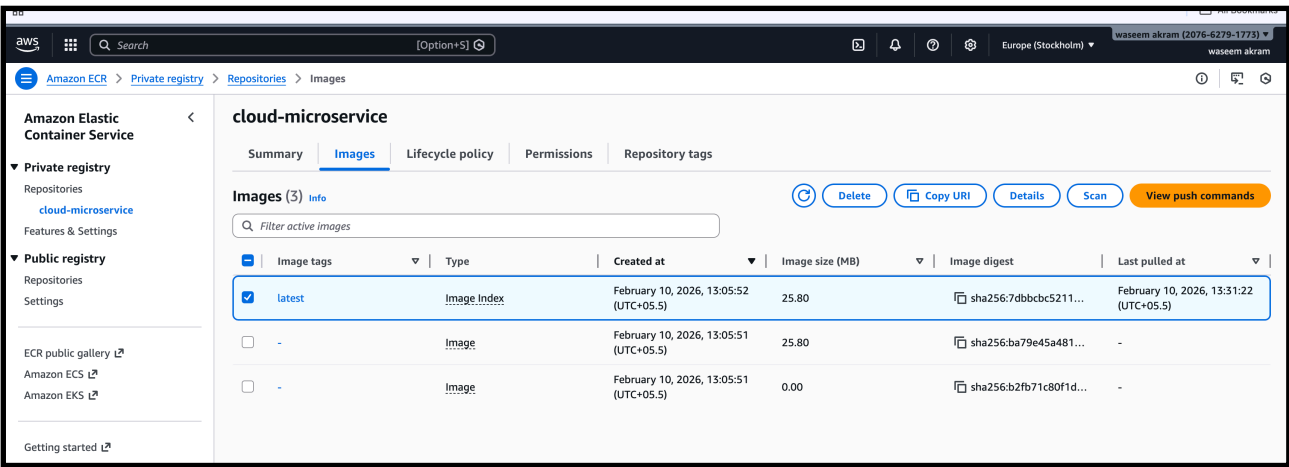
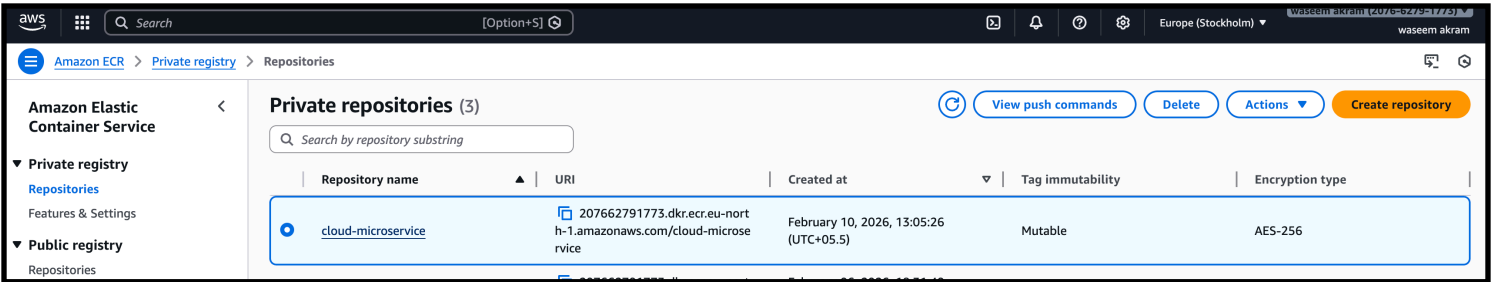
Docker Image Registry (ECR)

AWS Console → ECR → Repositories

Show:

- Repository: `cloud-microservice`
- Image present (`latest`)

Say:



Docker ps :

```
066e2fd751479a0c9166382eb525964460828/ba7128050617e506e4c89ea4dd
[root@ip-10-0-1-197 ~]# docker ps
CONTAINER ID   IMAGE                                     COMMAND                  CREATED        STATUS        PORTS                    NAMES
066e2fd75147   207662791773.dkr.ecr.eu-north-1.amazonaws.com/nginx-app:latest "/docker-entrypoint..." 14 seconds ago Up 13 seconds    0.0.0.0:80->80/tcp, :::80->80/tcp   pra
stine
[root@ip-10-0-1-197 ~]# aws ecr list-images --repository-name cloud-microservice
```

Verify:

Application Working (Browser)

Show this **FIRST**

- Open browser
- Paste ALB DNS:

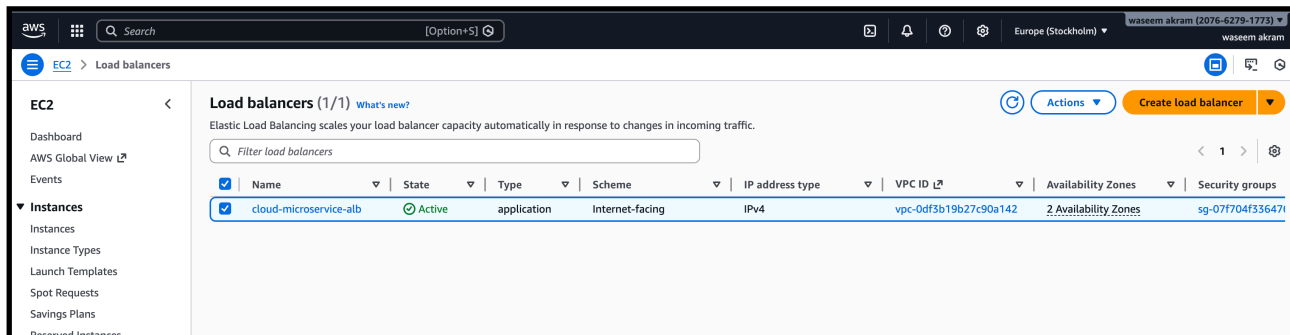


Load Balancer (Main Proof)

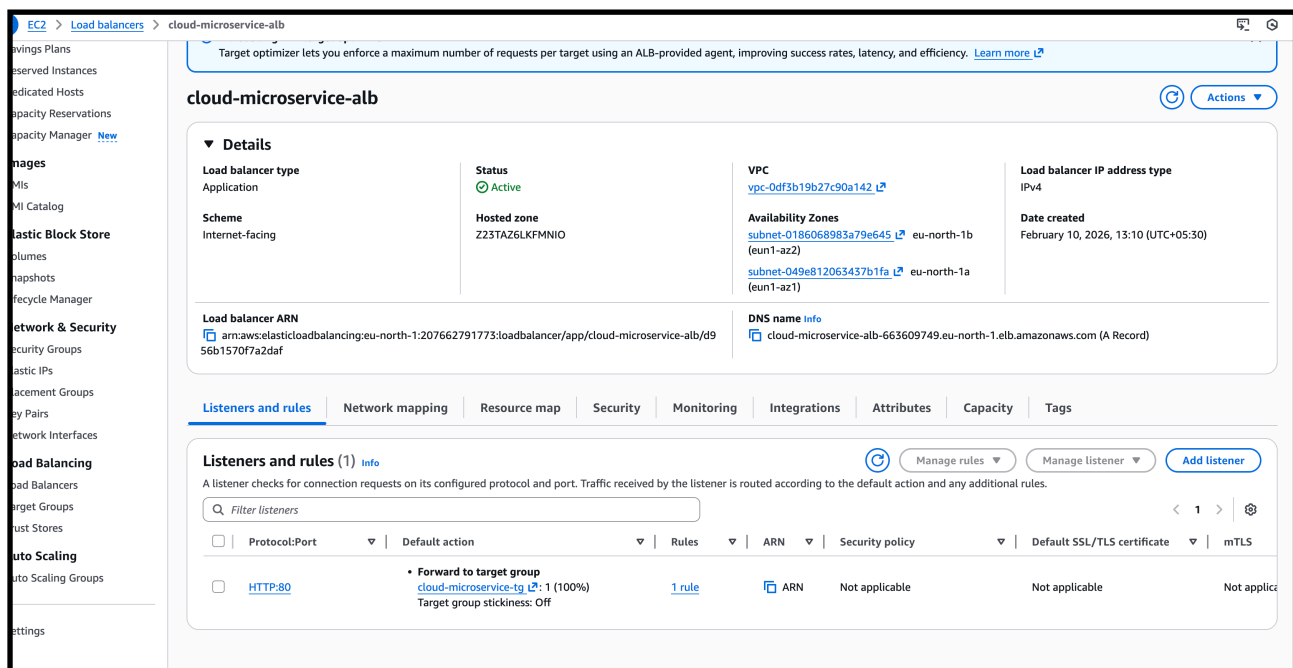
AWS Console → EC2 → Load Balancers

Show:

- Name: `cloud-microservice-alb`
- State: **Active**
- Type: **Application**
- Scheme: **Internet-facing**



Then click **Listeners**:



- Port **80**
- Forwarding to target group

Say:

“The ALB handles all incoming traffic.”

target Group (Health Proof)

EC2 → Target Groups → cloud-microservice-tg

Show:

- Target: **1 instance**
- Health status: **Healthy**

This is VERY important.

Say:

“The EC2 instance is registered and healthy.”

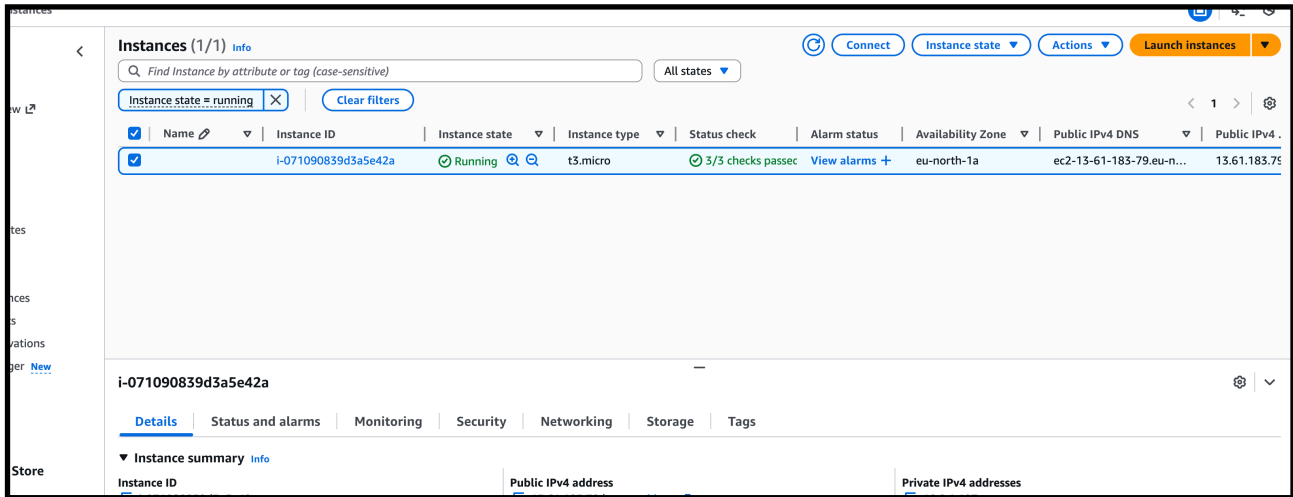
The screenshot shows the AWS Management Console interface for a Target Group named 'cloud-microservice-tg'. The left sidebar contains navigation links for various AWS services. The main content area displays the details of the target group, including its ARN, target type (Instance), IP address type (IPv4), protocol (HTTP), port (80), protocol version (HTTP1), and VPC ID. Below the details, there is a summary of target counts: 1 total target, 1 healthy, 0 unhealthy, 0 anomalous, 0 unused, 0 initial, and 0 draining. A section titled 'Distribution of targets by Availability Zone (AZ)' is also present. At the bottom, the 'Targets' tab is active, showing a table of registered targets. The table has columns for Instance ID, Name, Port, Zone, Health status, Health status details, Admini..., Overri..., and Launch... The first row shows a target with Instance ID 'i-071090839d3a5e42a', Name '-', Port 80, Zone 'eu-north-1a (e...', Health status 'Healthy', and Health status details '-'. The target is registered and healthy.

EC2 Instance (Single VM Proof)

EC2 → Instances

Show:

- Instance running
- Instance type: **t3.micro**
- Key pair: **ansible**
- IAM role attached



VPC & Subnets (Networking Proof)

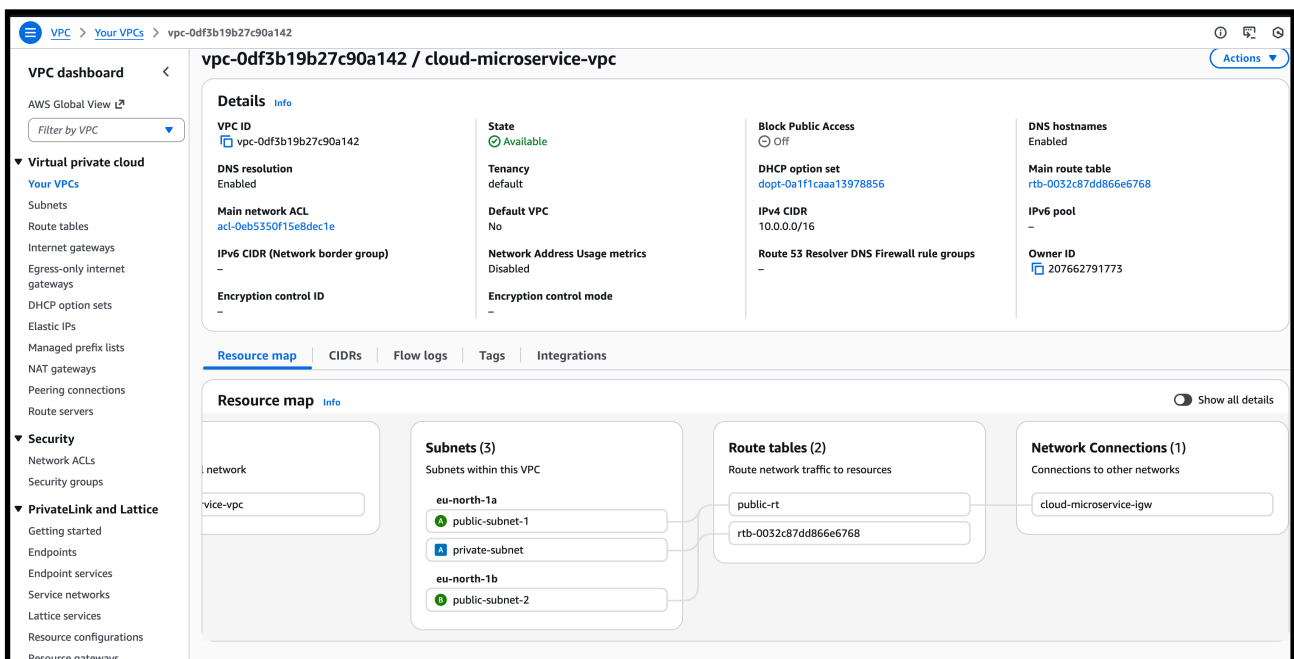
VPC → Your VPCs

- VPC: cloud-microservice-vpc

Subnets

- Public subnet (AZ-1)
- Public subnet (AZ-2)
- Private subnet (EC2)

Say:



ONE-SLIDE SUMMARY

If they ask “**Explain your setup**”, say:

“I used Terraform to deploy a Dockerized NGINX service on a single EC2 instance. The application is securely exposed through an Application Load Balancer, with images stored in ECR.”

Understood the Task

- Deploy a **simple microservice**
- Service should return a **static web page**
- Application must be **containerized**
- Image must be stored in a **Docker registry**
- Service must run on **one VM**
- App should be accessible **only via Load Balancer**
- Infrastructure must be created using **Terraform**

—> Created Project Structure

- Created separate folders for:
 - Docker files
 - Terraform infrastructure
- This helped keep **application code** and **infrastructure code** separate.

—> Dockerized the Application

- Used **NGINX** as the web server

- Created a simple `index.html` file
- Wrote a `Dockerfile` to:
 - Use NGINX base image
 - Copy static HTML file
 - Expose port 80
- Built the Docker image locally

—>Created Docker Registry (ECR)

- Used Terraform to create an **Amazon ECR repository**
- Tagged the local Docker image
- Pushed the image to ECR
- This registry is used later by EC2 to pull the image

—>Designed the Network (VPC)

- Created a **custom VPC**
- Created:
 - Two **public subnets** (for Load Balancer)
 - One subnet for EC2
- Added:
 - Internet Gateway
 - Route table and associations
- This was required so the ALB could work across **two Availability Zones**

—> Created Security Groups

- One security group for **Load Balancer**
 - Allowed HTTP traffic from the internet
- One security group for **EC2**
 - Allowed HTTP traffic **only from ALB**

- Allowed SSH access using key pair
- This ensured the EC2 instance is **not directly exposed**

—> Created IAM Role for EC2

- Created an IAM role for EC2
- Attached **ECR read-only policy**
- Attached role to EC2 instance
- This allowed EC2 to **pull Docker images securely** from ECR
- No hard-coded credentials were used

—> Created EC2 Instance

- Created **one EC2 instance** using Terraform
- Used **Amazon Linux 2023 AMI**
- Attached:
 - Security group
 - IAM role
 - SSH key pair
- Used **user-data script** to:
 - Install Docker
 - Start Docker service
 - Pull Docker image from ECR
 - Run the container

—> Handled User-Data & Docker Issues

- Docker was not initially installed due to OS differences
- Updated user-data to support **Amazon Linux 2023**
- Learned that **user-data runs only at instance creation**
- Forced EC2 replacement so updated script could run

- Verified Docker installation and container execution

—> **Created Application Load Balancer**

- Created an **Application Load Balancer**
- Attached it to **two public subnets**
- Created:
 - Listener on port 80
 - Target group
- Registered EC2 instance as a target
- Health checks confirmed instance was healthy

—> **Verified Application Access**

- Accessed the application using **ALB DNS name**
- Confirmed the static page loads successfully
- Verified that accessing EC2 public IP does **not** expose the app

—> **Final Verification (Visual)**

- Verified in AWS Console:
 - EC2 instance is running
 - ALB is active
 - Target group is healthy
 - ECR contains the Docker image
 - Security groups are correctly configured

—> **FINAL RESULT**

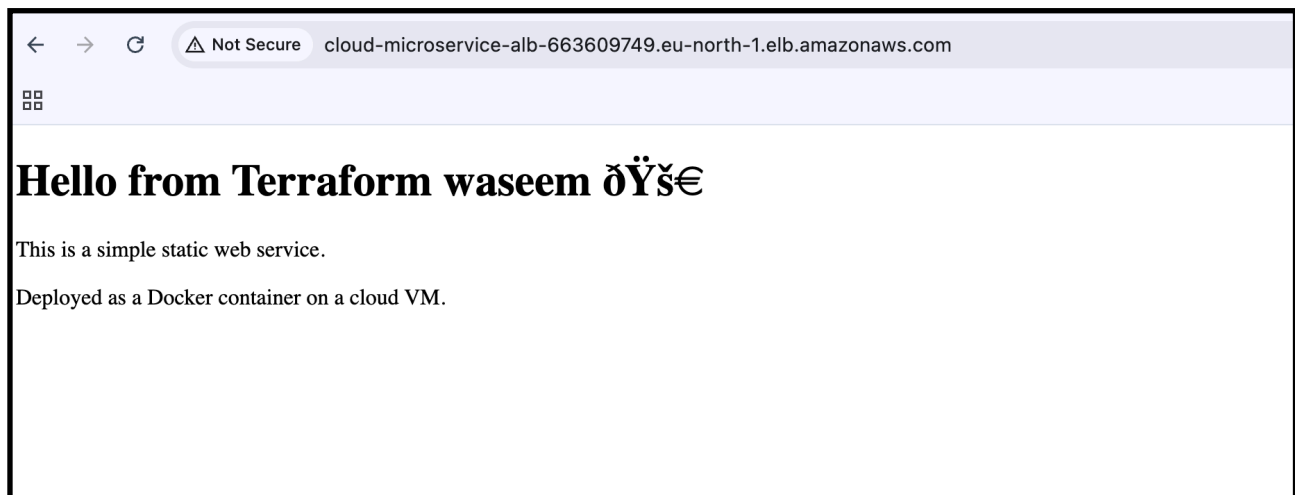
- Dockerized NGINX microservice deployed
- Image stored in Amazon ECR
- Service running on a single EC2 instance

- Application exposed only through ALB
- Entire infrastructure managed using Terraform

Below repo link :as all folder structure used in this task

<https://github.com/Waseema761/docker-terraform.git>

FINAL RESULT :



What this screenshot proves (in simple words)

- The **Application Load Balancer (ALB)** is running
- ALB is forwarding traffic to the **EC2 instance**
- EC2 is running a **Docker container**
- The container is using **NGINX**
- NGINX is serving a **static HTML page**
- The app is **NOT accessed directly**, only via ALB

--> alb is running and showing the containerised docker image running from nginx

--> and later if we want to destroy whole infrastructure we destroy by terraform destroy

```
aws_route_table_association.public_assoc_2: Destruction complete after 1s
aws_route_table.public_rt: Destroying... [id=rtb-0d9ce80f2932734e9]
aws_route_table.public_rt: Destruction complete after 2s
aws_internet_gateway.igw: Destroying... [id=igw-0b713a3932d01bd59]
aws_lb.this: Still destroying... [id=arn:aws:elasticloadbalancing:eu-north-1...loud-microservice-alb/903f142965f3babd, 00m10s elapsed]
aws_instance.app: Still destroying... [id=i-03f1192f053a9d7df, 00m10s elapsed]
aws_internet_gateway.igw: Still destroying... [id=igw-0b713a3932d01bd59, 00m10s elapsed]
aws_lb.this: Destruction complete after 18s
aws_subnet.public_2: Destroying... [id=subnet-0694cbb91a5dc0da8]
aws_instance.app: Still destroying... [id=i-03f1192f053a9d7df, 00m20s elapsed]
aws_internet_gateway.igw: Still destroying... [id=igw-0b713a3932d01bd59, 00m20s elapsed]
aws_subnet.public_2: Destruction complete after 4s
aws_instance.app: Still destroying... [id=i-03f1192f053a9d7df, 00m30s elapsed]
aws_internet_gateway.igw: Destruction complete after 29s
aws_instance.app: Destruction complete after 32s
aws_subnet.public: Destroying... [id=subnet-07137a2b3d9c2cc2b]
aws_security_group.ec2_sg: Destroying... [id=sg-0fb798dfc9b0a6ad5]
aws_iam_instance_profile.ec2_profile: Destroying... [id=cloud-microservice-ec2-profile]
null_resource.docker_push: Destroying... [id=2307415222097457251]
null_resource.docker_push: Destruction complete after 0s
aws_ecr_repository.app: Destroying... [id=cloud-microservice]
aws_iam_instance_profile.ec2_profile: Destruction complete after 1s
aws_iam_role.ec2_role: Destroying... [id=cloud-microservice-ec2-role]
aws_subnet.public: Destruction complete after 1s
aws_ecr_repository.app: Destruction complete after 1s
aws_security_group.ec2_sg: Destruction complete after 2s
aws_security_group.alb_sg: Destroying... [id=sg-0ee71c3ca1cd27c86]
aws_iam_role.ec2_role: Destruction complete after 1s
aws_security_group.alb_sg: Destruction complete after 1s
aws_vpc.this: Destroying... [id=vpc-0f5f490c70a910ed5]
aws_vpc.this: Destruction complete after 1s

Destroy complete! Resources: 20 destroyed.
waseemakram@waseem terraform %
```

